

Discrete Mathematics

Dr. Aamir Alaud Din

Topic 4: Boolean Algebra

Semester: Spring 2026

Objectives

By the end of this course you will be able to:

- Explain the basics of boolean algebra.
- Use rules of boolean algebra in computer applications.

Boolean Algebra: Introduction

- The circuits in computers and electronic devices have inputs, each of which is either a 0 or a 1, and produce outputs that are also 0s and 1s.
- Circuits can be constructed using any basic element that has two different states.
- Such elements include switches that can be in either the on or the off position and optical devices that can be either lit or unlit.
- In 1938 Claude Shannon showed how the basic rules of logic, first given by George Boole in 1854 in his ***The Laws of Thought***, could be used to design circuits.
- These rules form the basis for Boolean algebra.

Boolean Algebra: Introduction

- In this topic we develop the basic properties of Boolean algebra.
- The operation of a circuit is defined by a Boolean function that specifies the value of an output for each set of inputs.
- The first step in constructing a circuit is to represent its Boolean function by an expression built up using the basic operations of ***Boolean Algebra***.
- We will provide an algorithm for producing such expressions.
- The expression that we obtain may contain many more operations than are necessary to represent the function.
- Later in the chapter we will describe methods for finding an expression with the minimum number of sums and products that represents a Boolean function.

Boolean Functions: Introduction

- Boolean algebra provides the operations and the rules for working with the set $\{0, 1\}$.
- Electronic and optical switches can be studied using this set and the rules of Boolean algebra.
- The three operations in Boolean algebra that we will use most are complementation, the Boolean sum, and the Boolean product.
- The complement of an element, denoted with a bar, is defined by $\bar{0} = 1$ and $\bar{1} = 0$.

Boolean Functions: Introduction

- The Boolean sum, denoted by $+$ or by OR, has the following values:

$$1 + 1 = 1 \qquad 1 + 0 = 1 \qquad 0 + 1 = 1 \qquad 0 + 0 = 0$$

- The Boolean product, denoted by $\cdot$ or by AND, has the following values:

$$1 \cdot 1 = 1 \qquad 1 \cdot 0 = 0 \qquad 0 \cdot 1 = 0 \qquad 0 \cdot 0 = 0$$

- When there is no danger of confusion, the symbol $\cdot$ can be deleted, just as in writing algebraic products.
- Unless parentheses are used, the rules of precedence for Boolean operators are: first, all complements are computed, followed by all Boolean products, followed by all Boolean sums.

Boolean Functions: Introduction

Example

Find the value of $1.0 + \overline{(0 + 1)}$

Solution

$$\begin{aligned} 1.0 + \overline{(0 + 1)} &= 0 + \bar{1} \\ &= 0 + 0 \\ &= 0 \end{aligned}$$

Boolean Functions: Introduction

- The complement, Boolean sum, and Boolean product correspond to the logical operators, $\neg$, $\vee$, and $\wedge$, respectively, where 0 corresponds to F (false) and 1 corresponds to T (true).
- Equalities in Boolean algebra can be directly translated into equivalences of compound propositions.
- Conversely, equivalences of compound propositions can be translated into equalities in Boolean algebra.
- We will see later in this section why these translations yield valid logical equivalences and identities in Boolean algebra.
- Example below illustrates the translation from Boolean algebra to propositional logic.

Boolean Functions: Introduction

Example

Translate $1. 0 + \overline{(0 + 1)} = 0$, into a logical equivalence.

Solution

We obtain a logical equivalence when we translate each 1 into a T , each 0 into an F , each Boolean sum into a disjunction, each Boolean product into a conjunction, and each complementation into a negation. We obtain

$$(T \wedge F) \vee \neg(T \vee F) \equiv F$$

Boolean Functions: Introduction

Example

Translate the logical equivalence $(T \wedge T) \vee \neg F \equiv T$ into an identity in Boolean algebra.

Solution

We obtain an identity in Boolean algebra when we translate each T into a 1, each F into a 0, each disjunction into a Boolean sum, each conjunction into a Boolean product, and each negation into a complementation. We obtain

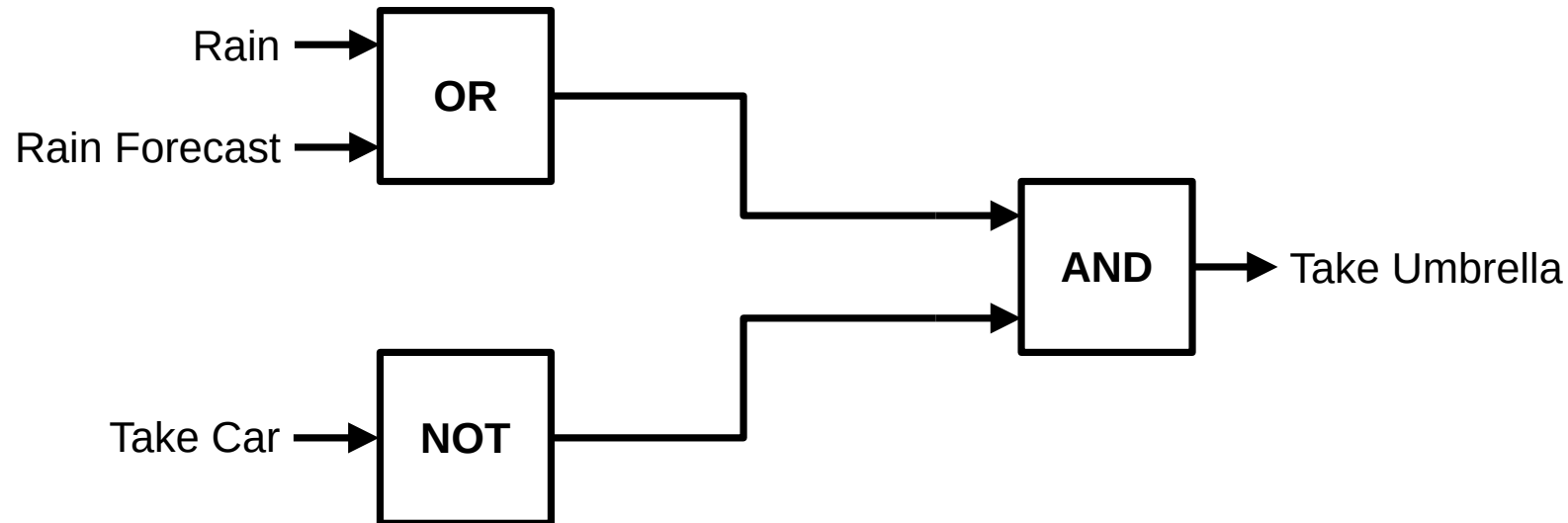
$$(1.1) + \bar{0} = 1$$

Introduction

Definition: Boolean Algebra

It is the set of rules, used to simplify the given logic expression without changing its functionality.

- It is used when the number of variables are less (2 ~ 3).



Introduction

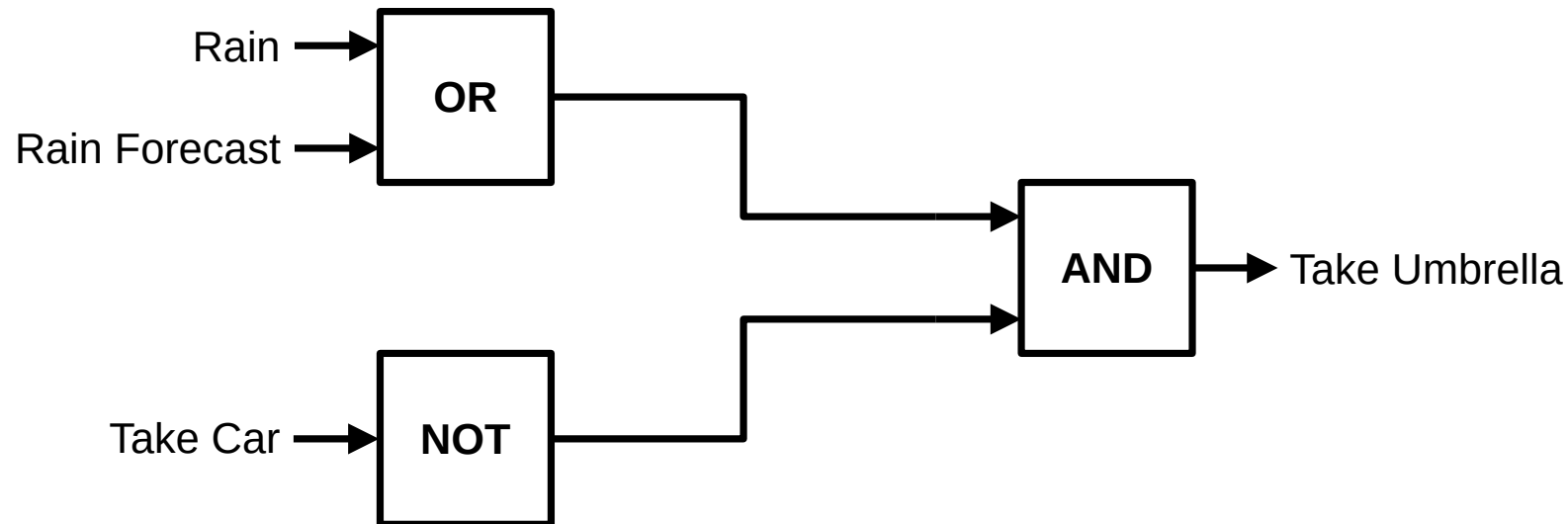
- Rules are used to simplify a logic expression without changing functionality.
- $F = A'B + BC + ABC$ can be simplified to $F = A'B + BC$.
- It is the drawback of boolean algebra that we need to verify the output using truth tables.
- Simplification of logic expression leads to different functions and the only way to check equivalency is to use the truth table.
- If number of variables are less (1 ~ 3), simplification is easy.
- With increasing number of variables (4 and more), simplification is complex and tedious which is the drawback of boolean algebra.

Introduction

- For higher number of variables, we use another method called the maps or Karnaugh maps.
- Note that $A'B + BC + ABC$ involves one NOT gate, three AND gates, and two OR gates.
- The expression $A'B + BC$ involves one NOT gate, two AND gates, and one OR gates.
- This simplification reduces the price of one gate and industry saves billions of dollars for trillions of circuits.
- This also reduces the power dissipation across gates reducing power consumption.

Introduction

- In the logic circuit below, take umbrella is the output (Y).
- Three inputs are rain (A), rain forecast (B), and take car (C).



Introduction

- From the above diagram, the logic expression becomes.

$$Y = (A + B).C'$$

- The same expression can also be expressed as

$$Y = BAC' + B'AC' + BC'$$

- The above expression can be simplified into the simpler upper version.

Introduction: Rules

1) Complement

- Complement of A is represented by A' .

$$0' = 1$$

$$1' = 0$$

- $(A')' = A$ and therefore $(0')' = (1)' = 1' = 0$.

Introduction: Rules

2) AND

- For two inputs A and B and one output Y , there will be $2^2 = 4$ outputs as shown in table below.

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Introduction: Rules

- There are four rules for the input A where A can be 0 or 1.

1. For $A = 0$ or $A = 1$

$$A.A = A$$

2. For $A = 0$ or $A = 1$

$$A.0 = 0$$

3. For $A = 0$ or $A = 1$

$$A.1 = A$$

4. For $A = 0$ or $A = 1$

$$A.A' = 0$$

Introduction: Rules

3) OR

- For two inputs A and B and one output Y , there will be $2^2 = 4$ outputs as shown in table below.

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Introduction: Rules

- There are four rules for the input A where A can be 0 or 1.

1. For $A = 0$ or $A = 1$

$$A + A = A$$

2. For $A = 0$ or $A = 1$

$$A + 0 = A$$

3. For $A = 0$ or $A = 1$

$$A + 1 = 1$$

4. For $A = 0$ or $A = 1$

$$A + A' = 1$$

Introduction: Rules

4) Distributive Law

$$A \cdot (B + C) = A \cdot B + A \cdot C$$

$$A + (B \cdot C) = (A + B) \cdot (A + C)$$

- Using second distributive law,

$$\begin{aligned} A + A'B &= A + A' \cdot B \\ &= (A + A') \cdot (A + B) \\ &= 1 \cdot (A + B) \\ &= A + B \end{aligned}$$

- Simplify $A' + AB$ to $A' + B$.

Introduction: Rules

5) Commutative Law

$$A + B = B + A$$

$$AB = BA$$

6) Associative Law

$$(A + B) + C = A + (B + C)$$

$$(AB)C = A(BC)$$

7) De Morgan's Laws

$$(A + B)' = A'B'$$

$$(AB)' = A' + B'$$

Introduction: Rules

Hierarchy

1. NOT

2. AND

3. OR

- Simplify $BAC' + B'AC' + BC'$ to $(A + B)C'$.

$$\begin{aligned} BAC' + B'AC' + BC' &= (B + B')AC' + BC' \\ &= 1 \cdot AC' + BC' \\ &= AC' + BC' \\ &= (A + B)C' \end{aligned}$$

Introduction: Rules

Example 1

Simplify $AB + AB'$

Solution

$$\begin{aligned} AB + AB' &= A(B + B') \\ &= A.1 \\ &= A \end{aligned}$$

NOTE

Prove the identity $AB + AB'$ using truth table.

Introduction: Rules

Example 2

Simplify $AB + ABC' + AB'C'$

Solution

$$\begin{aligned} AB + ABC' + AB'C' &= AB + AB'(C + C') \\ &= AB + AB'.1 \\ &= AB + AB' \\ &= A(B + B') \\ &= A.1 \\ &= A \end{aligned}$$

Introduction: Rules

NOTE

Prove the expression in Example 2 using truth table.

 **Thanks!**

Questions?